

Gradient UI Toolkit

User guide — creating and using custom GradientStyle assets with GradientElement

This package lets you paint smooth, customizable color gradients (linear or radial) as the background of any UI Toolkit element, using a **GradientStyle** asset and a **GradientElement** (<jc:GradientElement>) in your UXML/UI Builder hierarchy.

Step	Action
1	Create a new GradientStyle asset
2	Customize its colors, mode and angle
3	Reference it on a GradientElement in your UXML
4	Preview the result in UI Builder / Game view
5	(Optional) Enable the press/click gradient state

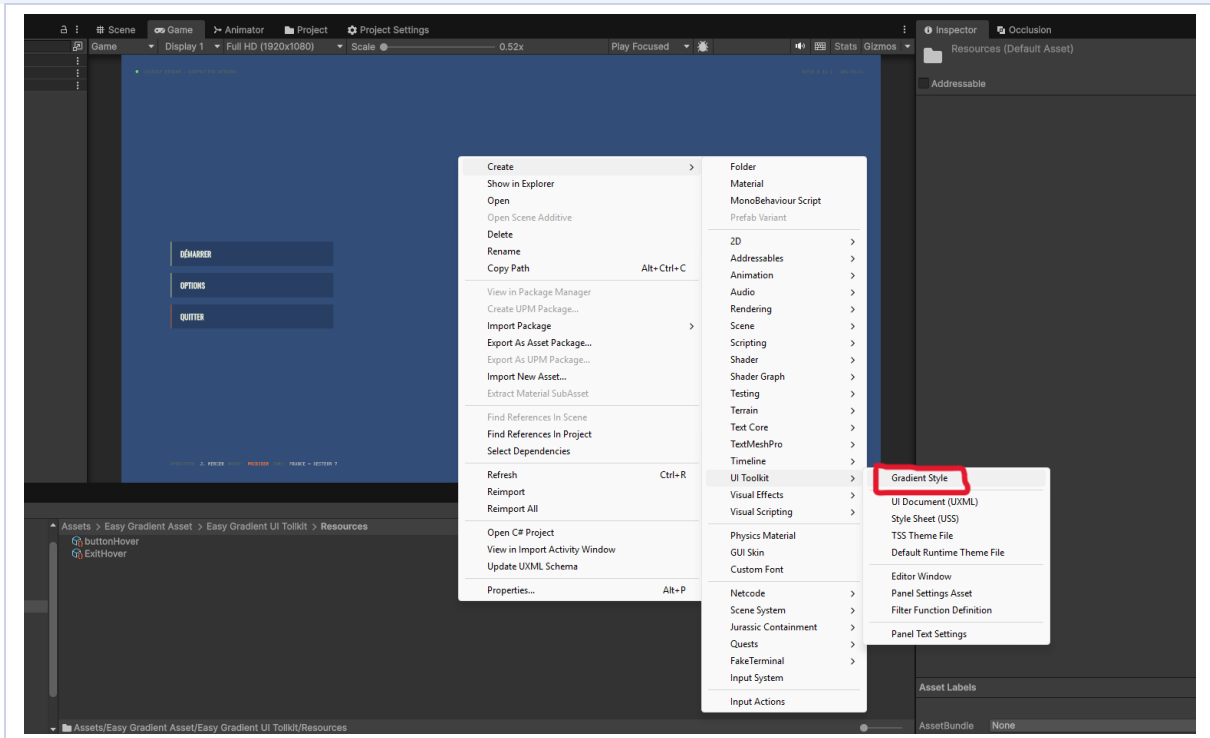
■ GradientElement works both by code (**GradientStyle** property) and by UXML (**gradient-style** / **gradient-style-hover** attributes), and supports **Linear** and **Radial** modes.

1

Create a new GradientStyle asset

In the **Project** window, right-click inside the folder where you want to store your gradient, then go to:

Create → UI Toolkit → Gradient Style



Create menu → UI Toolkit → Gradient Style

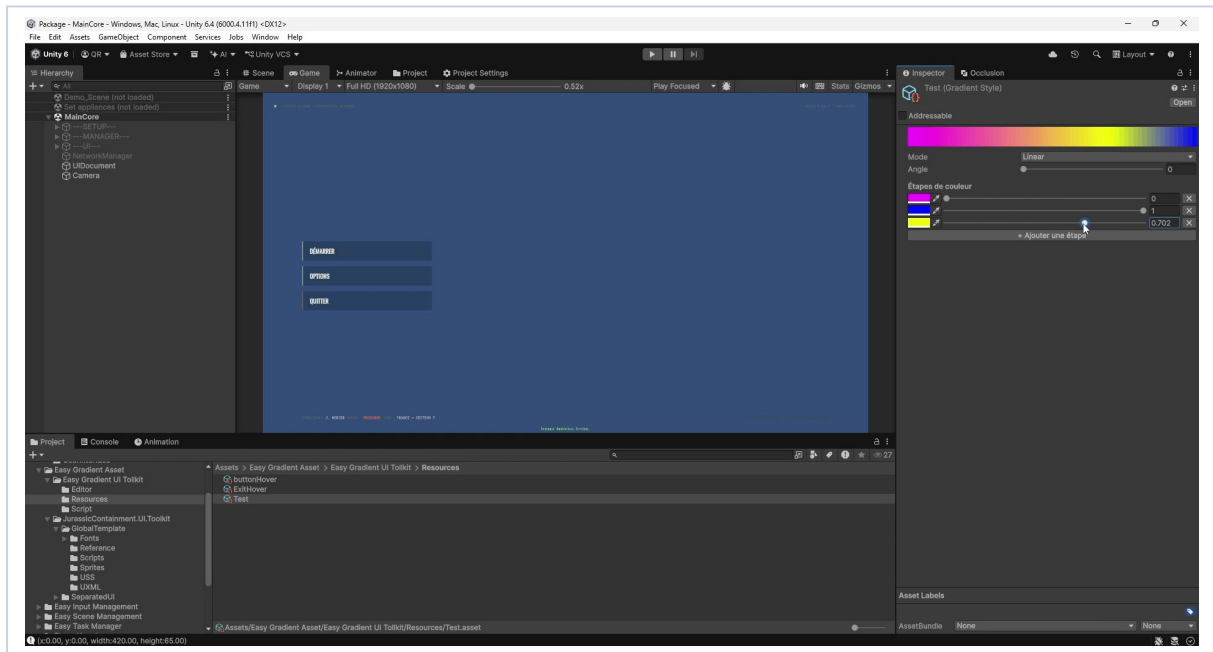
This creates a new **GradientStyle** ScriptableObject asset. Give it a clear, descriptive name right away — the name is what you'll reference later from UXML (via **Resources.Load**), so it must live under a **Resources** folder.

2

Customize the gradient

Select the new asset. Its custom inspector shows a live preview strip of the gradient, plus the following properties:

Mode	Linear paints bands along an axis; Radial paints concentric rings from the element's center.
Angle	Direction of the Linear gradient, in degrees (0° = left → right, clockwise). Ignored in Radial mode.
Color Stops	Each stop has a color and a position (0-1) along the gradient. Use + Add Stop to add one, and the X button to remove one (at least one stop must remain).



Custom GradientStyle inspector — Linear mode, 3 color stops (magenta → blue → yellow)

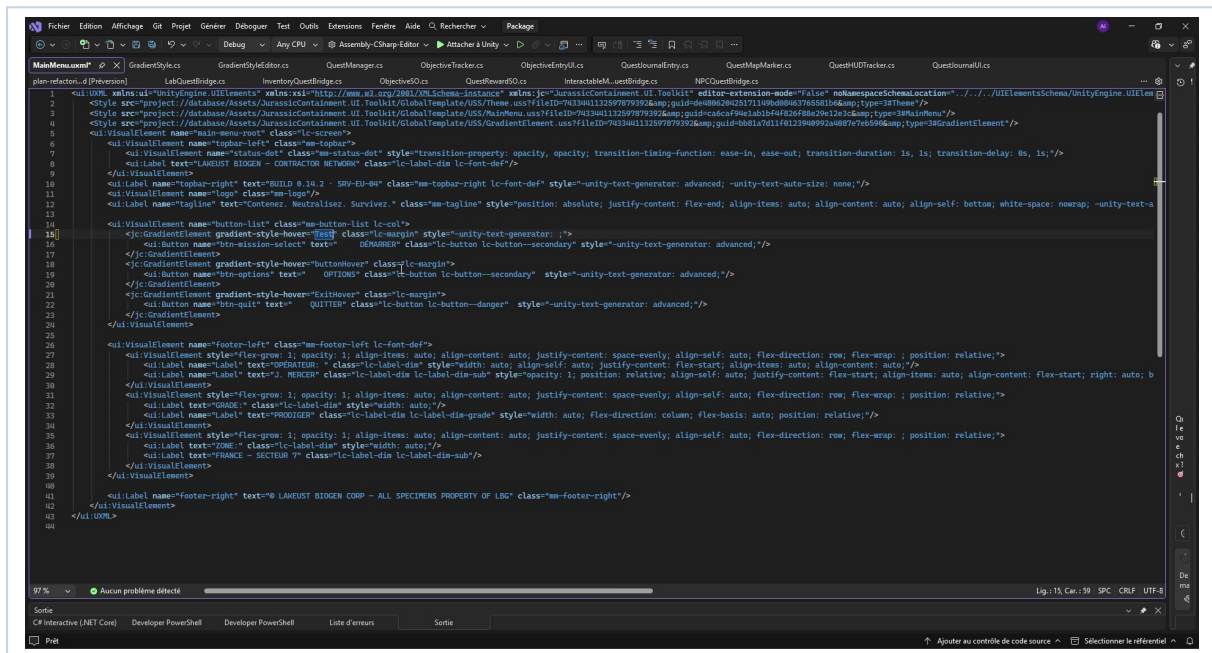
Wrap the element you want to give a gradient background to with a `<jc:GradientElement>` tag, and set one of its attributes to the **Resources**-relative path of your GradientStyle asset:

gradient-style

Base gradient, always shown (unless a hover/press gradient overrides it).

gradient-style-hover

Optional gradient shown while the pointer is hovering the element.



Setting the `gradient-style-hover` attribute to the name of your GradientStyle asset (here, the asset named "Test")

■ The value you type must match the asset's **file name** (or its path under Resources/), not the C# variable name. Rename the asset first if you want a clearer reference.

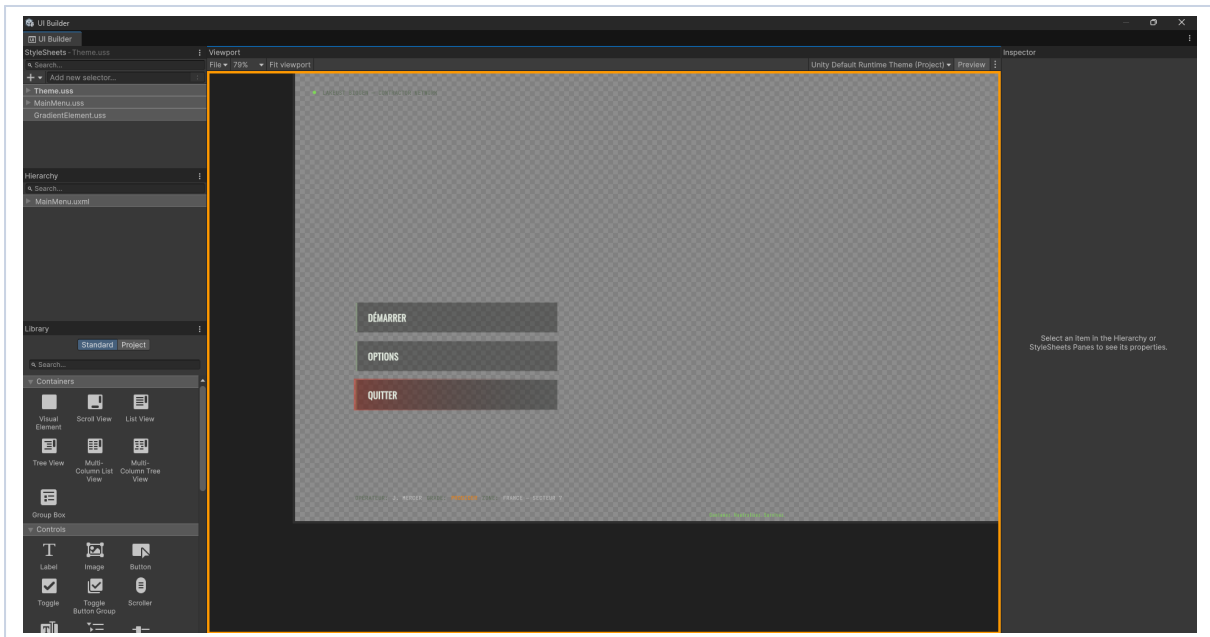
In the full UXML, the buttons of a menu are each wrapped in their own `<jc:GradientElement>`, each pointing to a different GradientStyle:



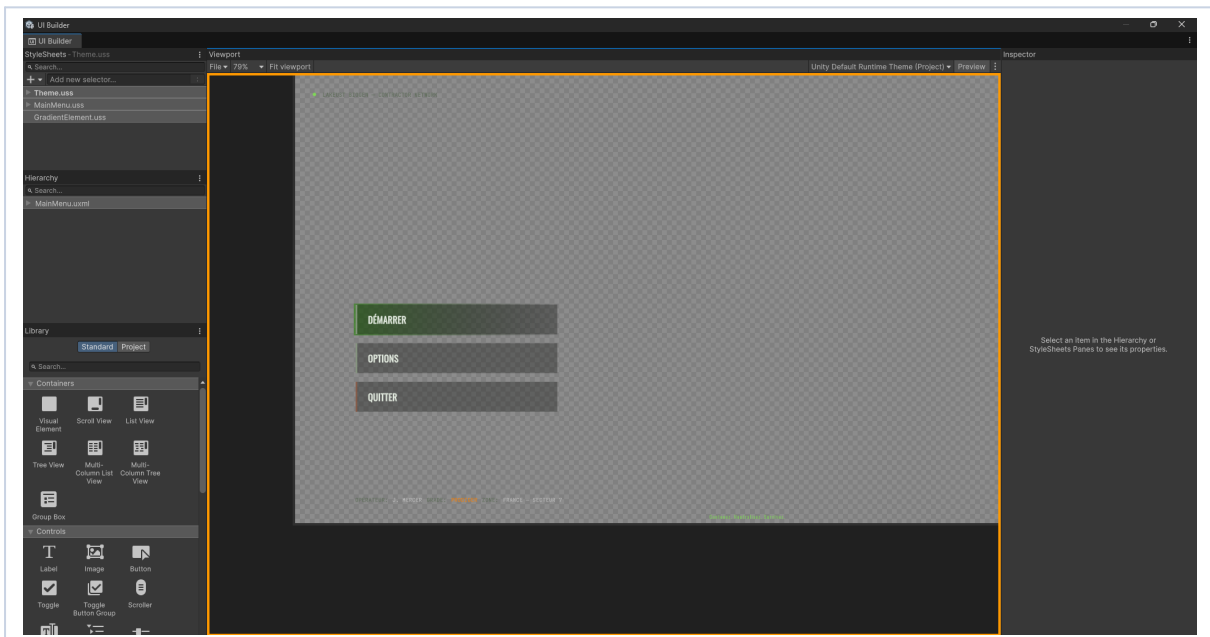
4

Preview the result

Open the UXML in **UI Builder** (or press Play) to see the gradients rendered. With the default per-button hover gradients, hovering (or selecting, in the Builder preview) each button reveals its own tint — for example green on Start, red on Quit:

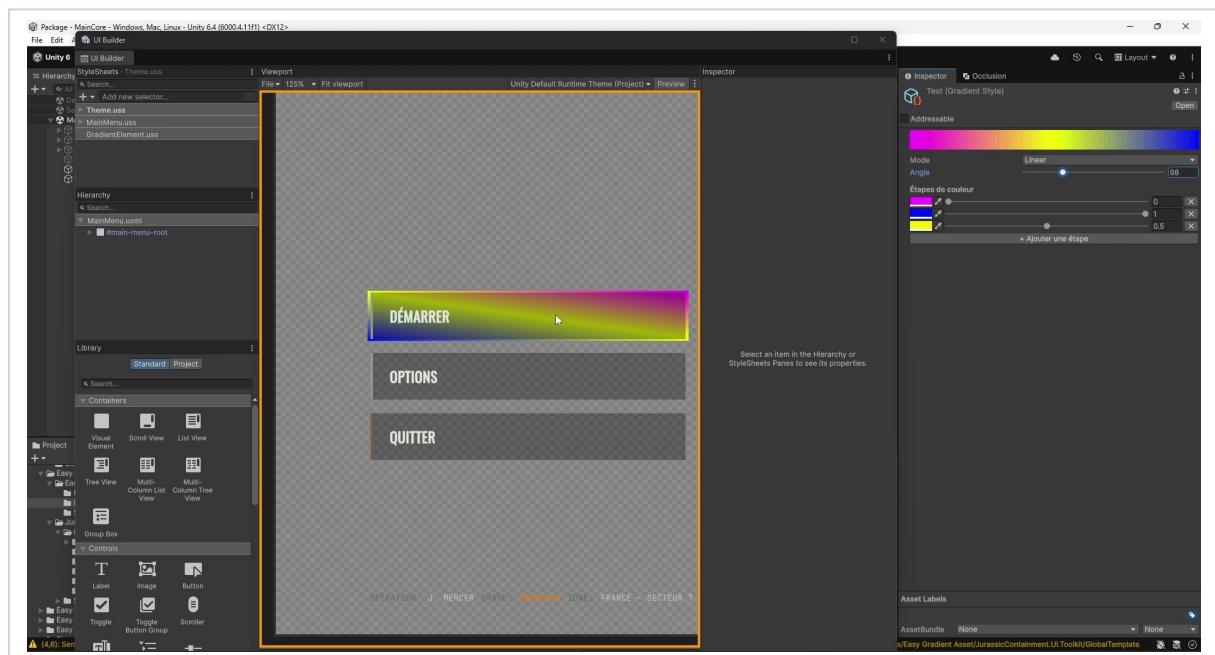


Default hover gradients previewed in UI Builder



Same menu, default state

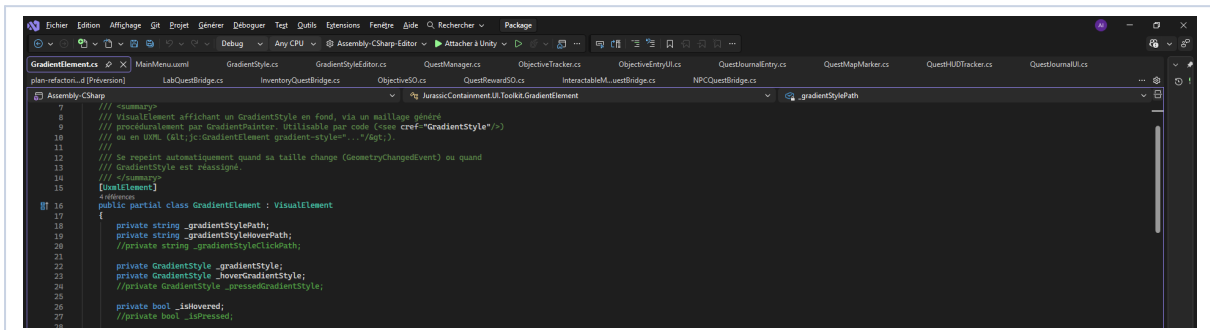
Once the button is pointed at your new custom GradientStyle asset (from Step 2), it renders your exact colors, mode and angle — here, the Linear magenta → blue → yellow gradient at a custom angle:



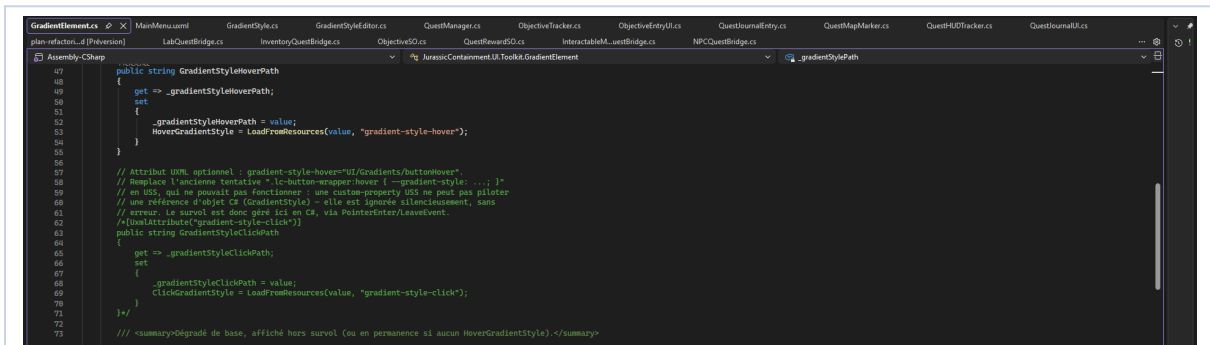
Custom GradientStyle applied to the Start button, with live angle/stops editing in the Inspector

GradientElement ships with a third, optional gradient slot for the pressed/clicked state (**gradient-style-click**), matching the pattern of the hover attribute. It is currently **commented out** in the script, since it isn't needed by default. To enable it:

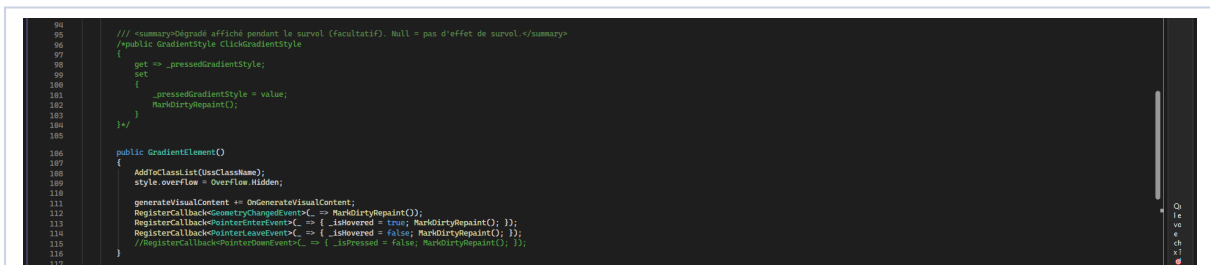
- 1 Open **GradientElement.cs** in your code editor.
- 2 Uncomment the private field **_gradientStylePressPath**, the **_pressedGradientStyle** field and the **_isPressed** field.
- 3 Uncomment the **GradientStylePressPath** UXML attribute property (**[UxmlAttribute("gradient-style-click")]**).
- 4 Uncomment the **PressGradientStyle** public property.
- 5 Uncomment the **PointerDownEvent** callback registration in the constructor.
- 6 Uncomment the **_isPressed** branch inside **OnGenerateVisualContent**'s switch expression, so the pressed gradient takes priority when set.



GradientElement.cs — commented-out press fields and property to uncomment



GradientElement.cs — commented-out gradient-style-click UXML attribute to uncomment



GradientElement.cs — commented-out PressGradientStyle property and PointerDownEvent callback to uncomment

■ Once uncommented, you can add a **gradient-style-click** attribute in UXML the same way you added **gradient-style-hover**, pointing to the GradientStyle asset you want shown while the element is pressed.

Quick recap

- 1 Create → UI Toolkit → Gradient Style, save it under a Resources folder
- 2 Set its Mode (Linear/Radial), Angle and Color Stops in the Inspector
- 3 Wrap your element with `<jc:GradientElement gradient-style="..." />`
- 4 Optionally add `gradient-style-hover="..."` for a hover state
- 5 Preview in UI Builder or Play mode
- 6 Optionally uncomment the press/click code path in `GradientElement.cs` for a 3rd state

Package scripts

`GradientStyle.cs` (data asset) · `GradientStyleEditor.cs` (custom inspector) · `GradientElement.cs` (UI Toolkit `VisualElement`) · `GradientPainter.cs` (mesh generation)

Guide generated for the "Gradient UI Toolkit" package (`JurassicContainment.UI.Toolkit`).